

How many EVs can the network support?

The benchmark records **65,544** daily charging sessions with LLF on the normal June scenario. Under the January +20% stress scenario, LLF records 26,553. These are the highest citywide passing fleets among the seven tested strategies.

Charging strategy	June: passing fleet	January +20%: passing fleet
Immediate	13,342	3,173
Fixed delay	7,907	2,342
Randomized delay	17,543	8,933
Capacity-aware	60,311	24,126
Least laxity first (LLF)	65,544	26,553
Valley filling	61,073	25,900
Voltage responsive	62,041	22,843

A vehicle counts as served only when its full charging request is delivered by departure and every simulated interval meets the adopted grid limits. Each EV requests 14 kWh at the battery, uses a 7.4 kW charger with 90% efficiency, arrives between 17:00 and 21:00, and leaves at 09:00 the next day.

The benchmark uses the existing adopted network ratings, a 1.04 pu source-voltage setting and non-EV peak shifting to 220 MW where needed, preserving daily energy. It adds no modeled network assets. This answers the no-upgrade question **conditional on those operating measures being available**. The benchmark uses 76,000 MWh for June and 120,000 MWh for January; these are scenario inputs, not a literal transcription of every photo-table entry.

The assumed aggregate LV budget is 120 MW: 50% of non-EV node demand and all EV charging count against it. Full adopted ratings remain usable; the stated reserve is not withheld as a separate safety margin.

Evidence status: all 70 benchmark rows were recorded, but the final job validation failed because the implementation fingerprint changed during execution. The numbers above are retained passing results, not a fully validated final benchmark or a measured city capacity. One random seed, 41001, was used. [1, 2]

How does charging location change the result?

Local concentration changes both capacity and the best-performing strategy. With charging spread citywide, LLF leads. When all sessions are assigned to TELEP, valley filling leads with 8,025 vehicles in June and 4,728 in the winter stress scenario.

Strategy	Citywide June	TELEP June	TELEP January +20%
Immediate	13,342	2,304	808
Fixed delay	7,907	1,294	584
Randomized delay	17,543	2,784	1,723
Capacity-aware	60,311	5,878	3,215
Least laxity first (LLF)	65,544	5,983	3,280
Valley filling	61,073	8,025	4,728
Voltage responsive	62,041	6,332	3,019

Spare capacity elsewhere in the city cannot remove a local feeder or district constraint. The district experiment changes electrical placement while retaining the same residential charging windows and energy request. It supports a conclusion about concentration, rather than a measured comparison of home, workplace and public charging.

Charging setting	Effect to represent in the model
Residential	Evening arrivals can overlap household demand. Overnight parking gives the controller time to defer charging.
Workplace	Daytime availability moves charging to a different baseline period. Its benefit depends on the supplying feeder and departure time.
Public chargers	Short stays and concentrated higher-power demand can tighten local limits. Charger power and dwell time must be modeled together.

The saved benchmark does not contain separate workplace or public-session capacity searches. Their effects above are modeling considerations, not additional benchmark findings. Citywide and TELEP results should not be relabeled as those location types. [1, 3]

When does simultaneous charging become critical?

The tests identify the first failed fleet beside each passing boundary. Immediate and delayed policies cross an electrical limit. The managed policies protect the grid but eventually miss departure energy. Both outcomes limit the fleet that can be served, but only the first is an observed overload.

Strategy	Next failed fleet	EV peak at passing fleet (MW)	Reason next fleet fails
Immediate	13,343	49.30	Aggregate grid limit
Fixed delay	7,908	58.51	Aggregate grid limit
Randomized delay	17,544	69.42	Aggregate grid limit
Capacity-aware	60,312	86.11	Departure energy shortfall
Least laxity first (LLF)	65,545	86.35	Departure energy shortfall
Valley filling	61,074	75.30	Departure energy shortfall
Voltage responsive	62,042	86.11	Departure energy shortfall

A daily fleet is not a simultaneous full-power count. At time t , charging power is $P_{EV}(t) = \sum p_i(t)$. For identical chargers, $P_{EV}(t) / 7.4 \text{ kW}$ is the equivalent number operating at full power; a controlled fleet can contain more active cars drawing less power each.

For example, 65,544 chargers all drawing 7.4 kW would demand 485.0 MW from EVs alone. That is not what the LLF run does. The EV peaks in the table are extracted from the saved 132-interval passing traces, excluding non-EV demand. They describe the tested schedules, not a universal instantaneous overload threshold.

The time-dependent admission algorithm on page 5 tests a specified placement against the remaining capacity and AC constraints. Exact simultaneous limits require that placement and time. The saved fleet search alone cannot supply one universal critical count. A solver failure is also kept separate from a physical overload. [1, 3]

Which charging strategies reduce the impact?

The algorithms differ in when they charge, how they share limited capacity and how they respond to voltage. The benchmark includes all seven implementations below; no RL result is used.

Algorithm	Implemented charging rule
Immediate	Charge on arrival until the requested energy is delivered. This is the uncoordinated reference.
Fixed delay	Release home charging at 23:00. Moving everyone to the same start time can create a new peak.
Randomized delay	Add seeded start delays to spread charging after 23:00. It reduces synchronization without directly enforcing grid safety.
Capacity-aware	Prioritize departure needs within node, source and district budgets; reduce charging when the AC safety check requires it.
Least laxity first	Prioritize vehicles with the least time left after allowing for their required charging. The smoothed allocator can fall back to plain LLF.
Valley filling	Iteratively distribute charging across forecast low-demand intervals, with capacity projection and AC protection. It uses a persistence forecast.
Voltage responsive	Reduce power as observed local voltage falls, restore it gradually, and apply mandatory central grid protection. This is a custom voltage-droop heuristic.

For citywide charging, LLF serves 65,544 vehicles versus 13,342 for immediate charging, about 4.91 times as many under the same normal benchmark. For TELEP, valley filling gives the largest recorded bound. Fixed delay falls to 7,907 citywide vehicles: moving the peak is not necessarily reducing it.

At the June passing boundary, LLF used plain-LLF fallback in 56 intervals and valley filling used fallback in four. The rankings refer to these complete implementations, including fallback and grid protection.

At the matched 500-EV June test, all seven pass. The total supply peak is 145.45 MW for immediate charging and 143.58 MW for both delay policies, a 1.29% reduction. This small-fleet peak comparison and the maximum-fleet comparison answer different questions; peaks at unequal fleet sizes do not establish peak-reduction superiority.

Choice: use LLF as the leading citywide candidate and valley filling as the leading concentrated-district candidate in these results. Retain grid protection and deadline checks. The benchmark does not establish a universally best strategy. [1, 3]

How does the testing and admission algorithm work?

The implemented workflow combines a fleet-capacity search with time-step charging control. A separate instantaneous admission screen answers how much extra charging can be accepted now.

Step	Method
1. Define inputs	Load curve, network topology and ratings, source voltage, district capacities, vehicle locations, arrival/departure times, battery energy, charger power and efficiency.
2. Establish the baseline	Reconcile gross demand including losses with AC node loads. Check the zero-EV case before interpreting positive capacity.
3. Simulate a candidate fleet	Use the same seeded session pool across strategies. Advance 132 quarter-hour intervals, covering all departures; aggregate EVs at existing nodes.
4. Accept or reject	Require converged AC results, valid voltage/loading at every interval, and delivery of requested battery energy by departure. Distinguish grid failure, missed energy and numerical uncertainty.
5. Find a fleet boundary	Double the candidate count until a failure is bracketed, then refine the local transition to adjacent integers. Record the passing count and its next failed count.
6. Admit charging now	Include already committed EV loads. For a specified placement, increase proposed extra kW, test AC and capacity limits, refine the bracket, and recheck the safe point. Convert kW to full-power charger equivalents if needed.

Core constraints are $P_{\text{base}}(t) + P_{\text{EV}}(t) + \text{losses}$ within applicable supply limits; node voltages between 0.95 and 1.05 pu; line and transformer loading at most 100%; and $0.90 \sum [p_i(t) \times 0.25 \text{ h}]$ meeting each battery-energy request. P_{base} is reconciled net non-EV node demand, so losses are counted once. District and aggregate-stage constraints apply as well.

The solution uses Python, pandapower AC power flow, NumPy and bounded optimization routines. No MCP service or RL is needed to describe or execute these algorithms. Passing now does not guarantee energy by departure, so instantaneous admission must remain linked to scheduling. A one-hour stay at 7.4 kW and 90% efficiency can supply only 6.66 kWh, regardless of spare grid capacity.

[1] Saved benchmark results: bench-c0c965da89ab4b2d, 70 rows, seven strategies, ten tests; retained results subject to the final fingerprint failure stated on page 1. [2] Frozen suite: suite-19f52a8c3215ac54a015, regulated operation, seed 41001. [3] Project benchmark, controller and admission implementations; admission is documented separately and is not a rerun of the historical benchmark. Local evidence: artifacts/playground/full-benchmark-without-mpc; scripts/challenge_instant.py.